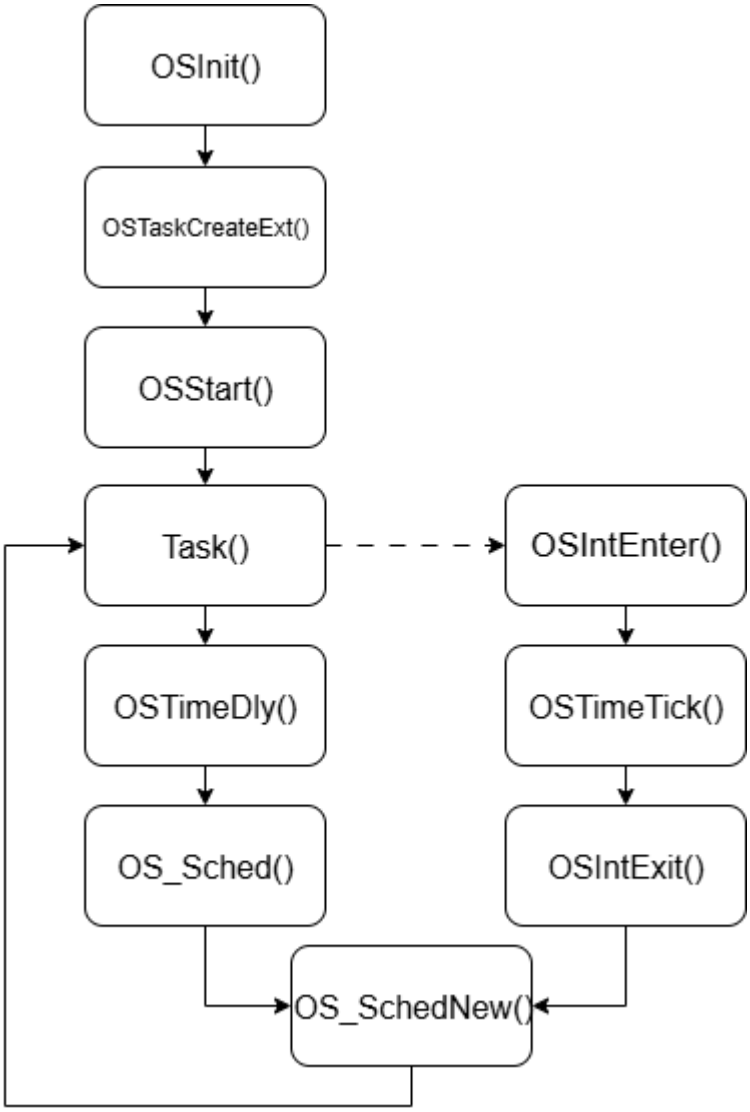
Embedded OS HW1

學號:M11407517 姓名:黃志傑

Problem Definition:

(a) Please draw the system flow of "Hello µC/OS-II (the modified main.c in Lab1)" and explain the process (functions). **Note: You should complete the provided diagram by filling in the blanks with µC/OS-II functions.**



- **OSInit() :**
  初始化 µC/OS-II 結構，包括**任務控制塊(TCB)**、**Ready Table**、時間管理 OSTimeSet 等。讓系統進入可建立任務的狀態。

- **OSTaskCreateExt() :**
  在 OS 初始化後、啟動多工前，你必須至少建立一個使用者任務。這個函式會為你的 Task() 函式準備好運行的環境，並將它放入「就緒列表」(Ready List) 中，等待被排程器(Scheduler)選中執行。

- **OSStart() :**
  啟動 Scheduler，從 Ready List 中選出最高優先權任務並開始執行且系統進入 multitasking 狀態。

- **Task() :**
  Task1、Task2 實際在做「工作」的地方。箭頭從 OS_SchedNew() 指向它，表示這個任務被排程器選中並開始(或繼續)執行。

- **OSTimeDly()**
  讓目前任務進入 Suspend status。
  將目前任務 (OSTCBCur) 的「延遲時間」設為 n ticks。呼叫 OS_Sched() 讓出 CPU，讓其他任務或 Idle Task 執行。

- **OS_Sched() :**

  從 Ready List 中，找出當前處於就緒狀態的任務中，優先級最高的那一個。當一個任務因為呼叫 OSTimeDly() 而進入等待狀態後，系統會呼叫排程器來決定下一個該由哪個任務來執行。

- **OS_SchedNew() :**

  負責執行實際的 Context Switch。在 OS_Sched() 找到新的最高優先級任務後，OS_SchedNew() 會儲存當前任務的 CPU 暫存器等狀態，並載入新任務的狀態，讓新任務開始執行。

- **OSIntEnter() : - 進入中斷**

  CPU 進入一個中斷服務程序(ISR)。它會增加一個全局的「中斷巢狀計數器」OSIntNesting。在流程圖中的角色：在任何 ISR 的最開頭，都應該先呼叫此函式。這能讓核心知道目前正處於中斷上下文，避免在 ISR 執行期間進行不安全的任務排程。

- **OSTimeTick() :**

  這個函式通常由一個週期性的硬體計時器中斷來呼叫。它是 uC/OS-II 時間管理的基礎。每呼叫一次，它會檢查所有因 OSTimeDly() 或其他帶有逾時設定的服務而等待的任務，將它們的延遲計數器減一。當有任務的計數器變為 0 時，該任務就會被移回 Ready List。

- **OSIntExit() :**

  ISR 的結尾呼叫。當 OSIntNesting 減至 0 時（表示所有巢狀中斷都已處理完畢），OSIntExit() 會自動呼叫排程器 OS_Sched()。因為中斷處理可能使得一個更高優先級的任務變為就緒狀態。如果新就緒的任務比被中斷的任務優先級更高，系統就會在中斷結束後，直接切換到那個更高優先級的任務去執行。它會接著跳到 OS_SchedNew() 進行 Context Switch。

(b) Consider two periodic tasks (τ_1 ,τ_2) and their delay time are 3 ticks and 4 ticks, respectively. Task priority of two tasks (τ_1 ,τ_2) are 1 and 2, respectively. Please add some code to the uCOS-II scheduler **in the kernel level** to observe how CPU is switched among tasks by means of context switches. **Note: You should add a printf statement to print "Task(ID) is running " message from within each task's main loop.**

- ucos_ii.h

```c
typedef struct task_para_set {
    INT16U TaskID;
    INT16U TaskArriveTime;
    INT16U TaskExecutionTIme;
    INT16U TaskPeriodic;
    INT16U TaskNumber;
    INT16U TaskPriority;
    INT16U job_no;                    //Job number
} task_para_set;
```

1. 在 task_para_set 結構增加計數 Job number 的 member

- app_hooks.c (APP_TaskSwHook())

```c
void  App_TaskSwHook(void)
{
#if (APP_CFG_PROBE_OS_PLUGIN_EN > 0) && (OS_PROBE_HOOKS_ENs > 0)
    OSProbe_TaskSwHook();
#endif
    if (start == 0) {
        printf("%d\t**********\ttask(%2d)(%2d)\t%d\n", OSTime,
            ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskID,
            ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->job_no,
            count);
        start++;
    }
    else  {
        if (OSPrioCur == 63) {
            printf("%d\ttask(%2d)\ttask(%2d)(%2d)\t%d\n", OSTime,
                63,
                ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskID,
                ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->job_no,
                count);
            count++;
        }
        else if (OSPrioHighRdy == 63) {
            printf("%d\ttask(%2d)(%2d)\ttask(%2d)\t%d\n", OSTime,
                ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskID,
                ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->job_no,
                63,
                count);
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->job_no++;
            count++;
        }
        else {
            printf("%d\ttask(%2d)(%2d)\ttask(%2d)(%2d)\t%d\n", OSTime,
                ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskID,
                ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->job_no,
                ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskID,
                ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->job_no,
                count);
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->job_no++;
            count++;
        }
    }
}
```

1. App_TaskSwHook 函式是一個由使用者定義的回呼函式 (Callback Function)，它會在 RTOS 核心每次進行任務切換 (Context Switch) 時被自動呼叫。
   - 事件時間：任務切換發生的系統時間 (OSTime)。
   - 切換來源：哪個任務被換出
   - 切換目標：哪個任務被換入
   - Counter：記錄總共發生了多少次任務切換。

2. 首次執行的特殊處理（if (start == 0)）

   系統啟動後，會從啟動程式碼切換到第一個使用者任務，這是第一次任務切換。

   由於這是第一次切換，還沒有「前一個任務」用 ********** 來表示。

   它只印出即將開始執行的新任務 (OSTCBHighRdy) 的 TaskID 和 job_no。

3. 根據是否有 idel task，printf 出對應的內容。

- main.c

```c
void task(void* p_arg) {
    task_para_set* task_data = p_arg;

    while (1) {
        printf("%d\ttask(%2d) is running\t\n", OSTime, task_data->TaskID);
        OSTimeDly(task_data->TaskPeriodic);
    }

}
```

1. 會印出「task(x) is running」以顯示目前執行的任務。
2. 會根據 TaskPeriodic 進行 OSTimeDly，將 task 狀態設為 suspend。